



90% of Security Vulnerabilities
Could Be Prevented!

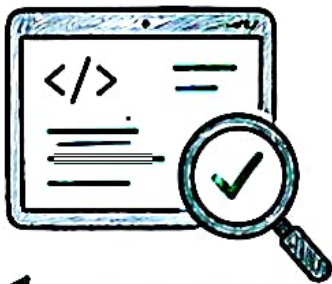


Do You Know When to Use
SAST vs DAST?



SAST VS **DAST**

➤ The DevSecOps Security Guide ➤



SCAN CODE
BEFORE RUN

- ✓ What is SAST?
- ✓ What is DAST?
- ✓ Key Differences
- ✓ Real CI/CD Pipeline
- ✓ Production Use Cases
- ✓ Interview Questions



SCAN RUNNING APP
LIKE A HACKER



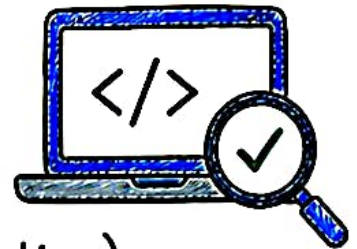
Swipe to learn which security test
catches vulnerabilities **BEFORE HACKERS DO!**



SLIDE 2



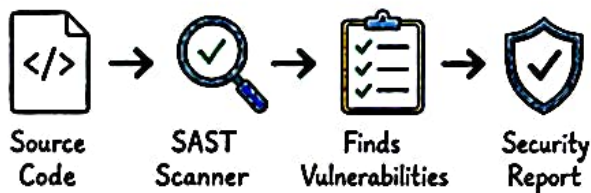
WHAT IS SAST?



(Static Application Security Testing)

SAST is a security testing method that analyzes source code, byte code or binaries without executing the application.

HOW IT WORKS



WHEN TO USE?



- During Development
- Before Build
- Before Deployment
- In CI/CD Pipeline

WHAT IT FINDS

- SQL Injection
- Hardcoded Secrets
- Cross Site Scripting (XSS)
- Weak Encryption
- Buffer Overflow
- Insecure Coding Practices
- ...and many more



Find issues early,
fix easily,
save cost
and time!

POPULAR SAST TOOLS



SonarQube



Checkmarx



Fortify



Semgrep

VERACODE

Veracode



Snyk Code



SAST helps you **SHIFT LEFT** and build secure applications from the start!

SLIDE 3



WHAT IS DAST ?

(Dynamic Application Security Testing)



DAST is a security testing method that analyzes a running application from the outside by simulating real world attacks, just like a hacker.

HOW IT WORKS



Deploy
Application



Application
Running



DAST
Scanner



Security
Report

≡ Application must be running ≡

WHEN TO USE?



- After Deployment
- Staging Environment
- QA Testing
- Production Validation (Carefully)
- Security Compliance

WHAT IT FINDS



• SQL Injection



• Cross-Site Scripting (XSS)



• Authentication Issues



• Session Management Problems



• Security Misconfigurations



• Sensitive Information Leakage



• Exposed APIs



• HTTP Security Header Issues

...and many more



DAST acts like a **real attacker** and finds issues that only appear when the app is **running!**

POPULAR DAST TOOLS



OWASP ZAP



Burp Suite



Acunetix



Invicti (Netsparker)



IBM AppScan



DAST helps you **validate security controls** by testing the real application like a hacker.





SAST vs DAST

QUICK COMPARISON



FEATURE	SAST	DAST
 WHAT IT TESTS	Source Code, Byte Code, Binaries	Running Application (Black Box Testing)
 APPLICATION RUNNING?	No ✗	Yes ✓
 TESTING APPROACH	Analyzes code statically without executing	Analyzes application dynamically like an attacker
 BEST TIME TO USE	During Development (Shift Left)	After Deployment in Staging / QA / Prod (Shift Right)
 FINDS	Coding Issues, Logic Flaws, Insecure Functions	Runtime Vulnerabilities, Config Issues, Logic Bypass
 USED BY	Developers	Security Engineers, QA, Pentesters
 SPEED	Fast 	Moderate 
 FINDS CODE LOCATION?	Yes ✓	No ✗
 EXAMPLES	SonarQube, Checkmarx, Semgrep, Fortify, Veracode	OWASP ZAP, Burp Suite, Acunetix, Invicti



BEST PRACTICE:

USE BOTH SAST & DAST FOR STRONGER APPLICATION SECURITY!



SAST = SHIFT LEFT (Prevent Issues Early)

DAST = SHIFT RIGHT (Detect Issues Late)



SLIDE 5

WHAT VULNERABILITIES DO THEY FIND?



Both find issues, but at **different stages!**



SAST

(Static Application Security Testing)

Finds issues in the CODE



- SQL Injection



- Hardcoded Secrets / Passwords



- Cross-Site Scripting (XSS)



- Weak Encryption / Cryptography



- Buffer Overflow



- Command Injection



- Path Traversal



- Insecure Coding Practices



- API Keys / Secrets in Code



- ...and many more

DAST

(Dynamic Application Security Testing)

Finds issues in the RUNNING APP



- SQL Injection



- Authentication Issues



- Authorization Flaws



- Session Management Problems



- Security Misconfigurations



- Exposed APIs



- HTTP Security Header Issues



- Sensitive Information Leakage



- Business Logic Flaws



- ...and many more



SAST finds problems in the CODE.

DAST finds problems in the RUNNING APPLICATION.

Use **BOTH** for Complete Security!



SLIDE 6



POPULAR TOOLS

For SAST & DAST



SAST TOOLS



SonarQube

Finds code smells, bugs, vulnerabilities & security issues.



Checkmarx

Static code analysis to find vulnerabilities early in the SDLC.



Fortify

Enterprise level SAST tool by Micro Focus.



Semgrep

Fast, lightweight static analysis for dev teams.

VERACODE

Veracode

Scans code, containers, APIs & IaC for security vulnerabilities.



Snyk Code

Developer friendly SAST for code & open source security.



CodeQL

Semantic code analysis engine by GitHub.



Other Tools

Mend, AppScan SAST, LGTM, RIPS, etc.

VS

DAST TOOLS



OWASP ZAP

Open source DAST tool from OWASP.



Burp Suite

Advanced web vulnerability scanner used by professionals.



Acunetix

Automated web vulnerability scanner.



Invicti (Netsparker)

Industry leading DAST with accurate results.



IBM AppScan

Enterprise DAST tool for web & API security.



Nikto

Open source web server scanner.



Vega

Open source scanner for web applications.



Other Tools

Wfuzz, Nmap, SQLMap, DirBuster, etc.



SAST tools scan your CODE.

DAST tools scan your RUNNING APPLICATION.



Choose tools based on your needs, technology stack & security goals!



SLIDE 7



SAST vs DAST

KEY DIFFERENCES & WHEN TO USE



SAST



1. Where it works
Analyzes source code, bytecode or binaries.



2. When to use
Early in SDLC (Design / Build phase)



3. Test type
White-box testing (Knows internal code)



4. Finds
Code vulnerabilities, logic flaws, insecure functions.



5. Advantage
Finds issues early, cheaper to fix, no app needed.



6. Limitation
May have false positives. Doesn't test running app.

VS

DAST



1. Where it works
Analyzes running application.



2. When to use
Later in SDLC (Test / Staging / Prod)



3. Test type
Black-box testing (No knowledge of code)



4. Finds
Runtime vulnerabilities, config issues, API flaws.



5. Advantage
Finds real-world issues, no code access needed.



6. Limitation
Finds fewer issues early, costly to fix later.

WHEN TO USE WHAT?



Early Development
Use SAST to catch issues in code.



Before Release
Use Both SAST + DAST for better coverage.



After Deployment
Use DAST regularly to find runtime vulnerabilities.



TAKEAWAY: Use SAST to build secure code.
Use DAST to keep running application secure.



SLIDE 8

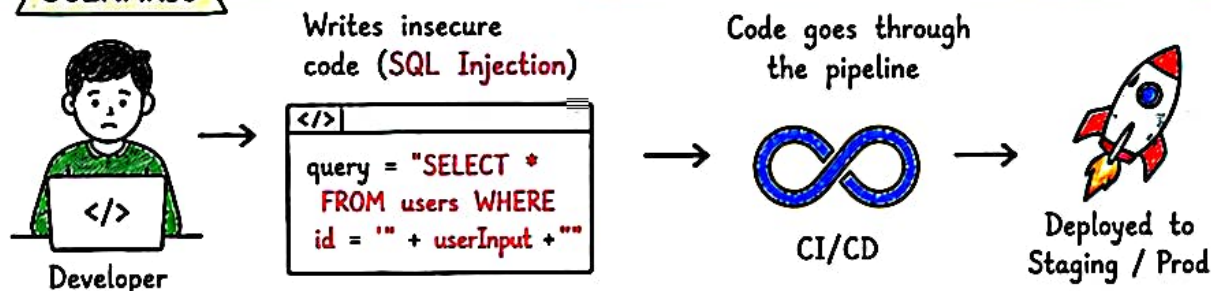


REAL PRODUCTION = SCENARIO =



Security is not a single check,
it's a CONTINUOUS process.

SCENARIO



SAST CATCHES EARLY



SAST tool scans the code before deployment.



Finds SQL Injection vulnerability in the code.



Issue fixed by developer before attackers can exploit.

IF
MISSED

DAST CATCHES AT RUNTIME



If SAST misses, DAST attacks the running application.



Finds SQL Injection vulnerability by simulating real attacks.



Security team fixes the issue before it causes damage.

THE LESSON



SAST finds problems in the CODE.

DAST finds problems in the RUNNING APPLICATION.

= TOGETHER THEY BUILD STRONG SECURITY! =



Secure Early. Validate Before Release. Protect Always.



SLIDE 9

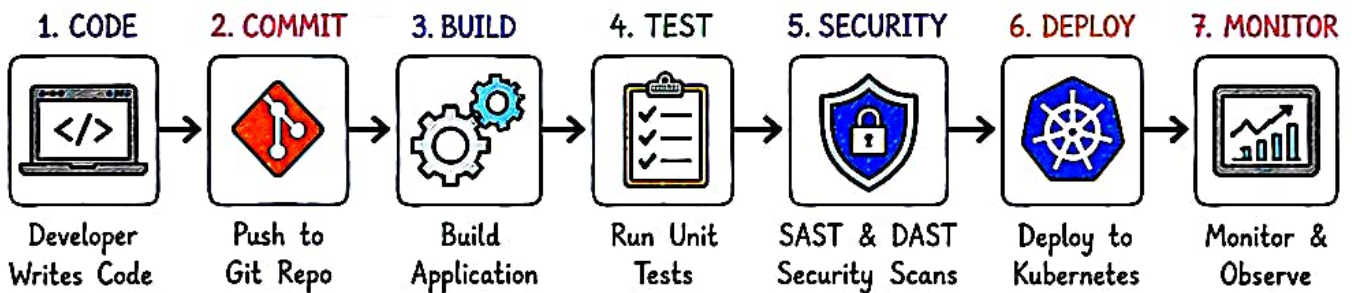


SAST vs DAST

in CI/CD PIPELINE



★ Shift Left with SAST. Shift Right with DAST.
Secure Every Step!



SAST (Shift Left)

- Runs during BUILD /CODE stage
- Analyzes source code, byte code or binaries
- Finds issues early before code runs
- Example Tools: SonarQube, Checkmarx, Fortify, Semgrep

Both
Together =
Complete
Security



DAST (Shift Right)

- Runs after DEPLOYMENT
- Tests the running application from outside
- Finds issues like auth, logic, configuration, API flaws
- Example Tools: OWASP ZAP, Burp Suite, Acunetix, Invicti

BENEFITS



Find issues
early & save
cost



Improve
code quality
& reliability



Reduce risk
in production
environment



Increase customer
trust & business
value



Meet compliance
& security
standards



Use SAST to prevent vulnerabilities.
Use DAST to detect what attackers might exploit.



Secure Code. Secure Runtime. Secure Future!



SLIDE 10



BEST PRACTICES SAST & DAST



Build Security In. Not After.



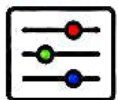
SAST BEST PRACTICES



- Scan early and often in the SDLC.



- Integrate SAST in CI pipeline.



- Configure rules based on your tech stack.



- Reduce false positives with proper tuning.



- Review & fix critical issues first.



- Educate developers on secure coding.



- Track metrics & improve over time.

DAST BEST PRACTICES



- Scan regularly in test & staging.



- Cover all endpoints, APIs & user flows.



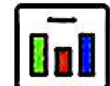
- Always test authentication, authorization & sessions.



- Use authenticated & unauthenticated scans.



- Prioritize & fix high risk vulnerabilities.



- Analyze reports & reduce noise.



- Rescan after fixes to ensure security.

SAST + DAST = MAX SECURITY



SAST finds issues
in CODE.



DAST finds issues
in RUNTIME.



Together they ensure
COMPLETE SECURITY!



Shift Left with SAST. Test Right with DAST.

@NeerajSingh-DevOps

